

# Liste e Pattern Matching

## Liste

In OCaml le liste sono un tipo di dato predefinito

Lista = *sequenza finita e immutabile di valori dello stesso tipo*

```
In [1]: let numeri = [3; 5; -1; 9; 14; 21] ;;
```

```
Out[1]: val numeri : int list = [3; 5; -1; 9; 14; 21]
```

Se gli elementi sono di tipo `XYZ`, il tipo della lista è `XYZ list`

Inoltre, la lista vuota si rappresenta semplicemente come `[]` e ha tipo generico:

```
In [2]: let lista_vuota = [] ;;
```

```
Out[2]: val lista_vuota : 'a list = []
```

Da questi esempi è evidente che una lista si rappresenta come una sequenza di valori racchiusi tra parentesi quadre e separati da punto e virgola. Il fatto che sia una sequenza *immutabile* significa che non sarà possibile modificare (aggiungere, rimuovere o modificare) gli elementi della lista. Ogni operazione che vedremo sulle liste potrà leggere ed elaborare gli elementi, e costruire nuove liste a partire da quelle esistenti, ma sempre senza possibilità di modificarle.

Le liste possono contenere elementi di qualunque tipo.

Una lista di stringhe:

```
In [3]: let stringhe = ["cane"; "gatto"; "rana"; "gnu"] ;;
```

```
Out[3]: val stringhe : string list = ["cane"; "gatto"; "rana"; "gnu"]
```

Una lista di tuple (tutte dello stesso tipo):

```
In [4]: let tuple = [ (1,"lun"); (2,"mar"); (3,"mer") ] ;;
```

```
Out[4]: val tuple : (int * string) list = [(1, "lun"); (2, "mar"); (3, "mer")]
```

Una lista di funzioni (tutte dello stesso tipo)

```
In [5]: let funzioni = [ (fun x -> x+1) ; (fun x -> x-1) ; (fun x -> x) ] ;;
```

```
Out[5]: val funzioni : (int -> int) list = [<fun>; <fun>; <fun>]
```

Nell'ultimo esempio, in realtà, gli elementi della lista (funzioni) non hanno tutti lo stesso tipo. Mentre le prime due hanno tipo `int -> int`, la terza ha tipo `'a -> 'a`.

Essendo però `'a -> 'a` compatibile con `int -> int` e più generico, i tipi delle tre funzioni possono essere unificati nel tipo più specifico `int -> int`. Approfondiremo questo aspetto tra poco.

Inoltre, è anche possibile creare liste di liste:

```
In [6]: let liste = [ [1; 2; 3] ; [1; 3; 2] ; [2; 1; 3] ; [2; 3; 1] ; [3; 1; 2] ; [3; 2; 1] ]
```

```
Out[6]: val liste : int list list =
  [[1; 2; 3]; [1; 3; 2]; [2; 1; 3]; [2; 3; 1]; [3; 1; 2]; [3; 2; 1]]
```

## Riflessione sul tipo di una lista

OCaml inferisce il tipo di una lista *unificando* i tipi dei suoi elementi:

```
In [7]: [] ;;
```

```
Out[7]: - : 'a list = []
```

```
In [8]: [[]; []] ;;
```

```
Out[8]: - : 'a list list = [[]; []]
```

```
In [9]: [[]; []; [3]] ;;
```

```
Out[9]: - : int list list = [[]; []; [3]]
```

```
In [10]: [[]; []; ["ciao"]] ;;
```

```
Out[10]: - : string list list = [[]; []; ["ciao"]]
```

Anche in questi esempi, come in quello della lista di funzioni visto poco sopra, si vede il meccanismo di unificazione dei tipi in opera. Le liste che contengono solo liste vuote hanno tipo (generico) `'a list list`. Non appena alle liste vuote si aggiunge una lista non vuota (di `int` e `string`, negli esempi) il tipo dell'intera lista viene reso più specifico (diventando `int list list` e `string list list`, rispettivamente).

Quello che fa il l'unificazione dei tipi è trovare il *tipo più generico possibile* che sia compatibile con tutti gli elementi della lista. Fintanto che nella lista vengono inserite solo liste vuote `[]` il tipo può rimanere generico (in quanto una lista vuota può essere di `int`, `string`, o qualunque altro tipo). Nel momento in cui nella lista venga incluso una lista non vuota (es. `[3]`), allora il tipo più generico possibile diventa il tipo di quella specifica lista, in quanto essa non può avere altri tipi se non quello dato dagli elementi che contiene (ad es. `int list`).

Con le funzioni:

```
In [11]: let f x y = 3;;
```

```
Out[11]: val f : 'a -> 'b -> int = <fun>
```

```
In [12]: let g x y = x+1 ;;
```

```
Out[12]: val g : int -> 'a -> int = <fun>
```

```
In [13]: let h x y = y+1 ;;
```

```
Out[13]: val h : 'a -> int -> int = <fun>
```

```
In [14]: let lista = [f ; g];;
```

```
Out[14]: val lista : (int -> 'a -> int) list = [<fun>; <fun>]
```

```
In [15]: let lista2 = h::lista;;
```

```
Out[15]: val lista2 : (int -> int -> int) list = [<fun>; <fun>; <fun>]
```

Questi esempi con le funzioni consentono di osservare nuovamente il funzionamento dell'unificazione dei tipi. Le tre funzioni `f`, `g` e `h` hanno tipi generici diversi. La lista `lista` ha un tipo che è il più generico possibile che sia compatibile con i tipi di `f` e `g`, ossia `(int -> 'a -> int) list`. Aggiungendo alla lista `h` (in `lista2`) il tipo deve essere ulteriormente specializzato per tenere conto del fatto che `h` ha un secondo parametro di tipo `int`.

È interessante vedere quale sia il tipo di una lista che contenga solo `g` e `h`:

```
In [16]: let lista3 = [g ; h] ;;
```

```
Out[16]: val lista3 : (int -> int -> int) list = [<fun>; <fun>]
```

Nonostante sia `g` che `h` abbiano un tipo polimorfo (generico), il fatto che `g` preveda un primo parametro di tipo `int` e `h` un secondo parametro di tipo `int` implica che la lista abbia tipo `int -> int -> int list`, cioè che si tratti di una lista di funzioni con due parametri *entrambi interi*.

Il tipo inferito da OCaml tramite il meccanismo di unificazione descritto da questi esempi, prende il nome di *most general unifier*, ossia (come già detto) è il tipo più generico possibile che tenga conto di tutti i vincoli imposti dai tipi che sono stati unificati.

## L'operatore cons ::

Una lista può essere *costruita* a partire da un'altra usando l'operatore `::` ("cons")

**Definizione ( :: ).** Data una lista `l` di tipo `T list` e un elemento `e` di tipo `T`, si denota con `e :: l` la lista in cui il primo elemento è `e` seguito dagli elementi in `l`

```
In [17]: let l1 = [3;2;1] ;;
         let l2 = 4 :: l1 ;;
```

```
Out[17]: val l1 : int list = [3; 2; 1]
```

```
Out[17]: val l2 : int list = [4; 3; 2; 1]
```

La notazione `[1; 2; 3; 4]` è in realtà "zucchero sintattico" per la notazione

```
In [18]: let lis = 1 :: (2 :: (3 :: (4 :: []))) ;;
```

```
Out[18]: val lis : int list = [1; 2; 3; 4]
```

che può essere scritta più semplicemente così (essendo `::` associativo a destra):

```
In [19]: let lis = 1 :: 2 :: 3 :: 4 :: [] ;;
```

```
Out[19]: val lis : int list = [1; 2; 3; 4]
```

Questa rappresentazione evidenzia la natura *induttiva* (incrementale) delle liste

- a partire dalla lista vuota `[]`, si concatena in testa `4`, poi `3`, poi `2`, poi `1`

### ATTENZIONE:

È comune dire che `::` *aggiunga* un elemento in testa alla lista, ma non è esatto:

- Le liste sono immutabili, quindi `e :: 1` concettualmente è una *lista diversa* con dentro `e` e gli elementi di `1`

Il fatto che le liste siano immutabili fa sì che OCaml non debba copiare gli elementi di `1` nella nuova lista

Vediamo perché...

Le liste in OCaml sono concepite come *liste concatenate singole*

La lista `lis1 = [2; 3; 4]` corrisponde a:



La lista `lis2 = 1 :: lis1` può essere ottenuta concatenando in testa:



Essendo immutabili (= no modifiche ai valori) per il programmatore è come se fossero due liste diverse

- no spreco di memoria

## Concatenazione di liste (append - @)

Date due liste `lis1` e `lis2`, l'operazione *append* `lis1 @ lis2` descrive la loro concatenazione in un'unica lista

```
In [20]: let lis1 = [1;2;3] ;;
         let lis2 = [4;5;6] ;;
         let lis3 = lis1 @ lis2 ;;
```

```
Out[20]: val lis1 : int list = [1; 2; 3]
```

```
Out[20]: val lis2 : int list = [4; 5; 6]
```

```
Out[20]: val lis3 : int list = [1; 2; 3; 4; 5; 6]
```

Internamente, la concatenazione crea una copia della prima lista

Date `lis1 = [1; 2; 3]` e `lis2 = [4; 5; 6]` :



Lista3

Ecco il risultato di `let lis3 = lis1 @ lis2 :`



Lista4

## Altre operazioni su liste (OCaml API)

Il modulo `List` dell'OCaml API (<https://ocaml.org/api/List.html>) fornisce moltissime funzioni per l'elaborazione di liste. Ad esempio:

```
In [21]: List.length [5;2;1] ;; (* Lunghezza della lista *)
```

```
Out[21]: - : int = 3
```

```
In [22]: List.hd [5;2;1] ;; (* head - primo elemento della lista *)
List.tl [5;2;1] ;; (* tail - elementi successivi al primo *)
```

```
Out[22]: - : int = 5
```

```
Out[22]: - : int list = [2; 1]
```

```
In [23]: List.rev [5;2;1] ;; (* rovescia la lista *)
```

```
Out[23]: - : int list = [1; 2; 5]
```

Interessante anche vedere il codice sorgente di tutte queste funzioni:

- <https://github.com/ocaml/ocaml/blob/trunk/stdlib/list.ml>

## Pattern Matching

Per accedere agli elementi di una lista è necessaria un'operazione di *destrutturazione*

- OCaml ha un potente meccanismo di *Pattern Matching* (non solo per liste)

**Sintassi:**

```
match EXP with
| P_1 -> EXP_1
| P_2 -> EXP_2
...
| P_N -> EXP_N
```

**Semantica informale:**

- il risultato di `EXP` viene confrontato con i pattern `P_1, ..., P_n`
- se `P_i` è il primo pattern con cui *fa match*, si valuta l'espressione `EXP_i`

## Sintassi dei pattern

Considerando i tipi visti fino ad ora (tipi di base, tuple e liste) la sintassi dei pattern è data da:

- valori di tipi base (non funzioni):  
`true, false, 0, 1, 2, 2.3, 4.5, 'a', 'b', 'c', "abc"`
- variabili: `x, y, z, ...`
- tuple: `(P_1, ..., P_N)`
- liste di lunghezza fissata: `[P_1, ..., P_N]`
- liste con *cons*: `P_1 :: P_2`
- wildcard: `_`

dove `P_1, ..., P_N` sono a loro volta dei pattern

**Definizione (Match)** Un valore `v` *fa match* con un pattern `P` se:

- `P = _`
- `P = v`
- esiste un modo di istanziare le variabili in `P` ottenendo `P'` tale che `P' = v`

Esempi:

| Pattern                         | Fanno match                                                | Non fanno match                             |
|---------------------------------|------------------------------------------------------------|---------------------------------------------|
| <code>1</code>                  | <code>1</code>                                             | <code>0, 2, 3</code>                        |
| <code>(3,2)</code>              | <code>(3,2)</code>                                         | <code>(2,3), (4,2)</code>                   |
| <code>(x,y)</code>              | <code>(3,2), (2,3), (4,2)</code>                           | <code>(1,3,2), (1,4,3,2)</code>             |
| <code>(x,2)</code>              | <code>(3,2), (4,2), ("ciao",2)</code>                      | <code>(2,3), (4,2,1)</code>                 |
| <code>[]</code>                 | <code>[]</code>                                            | <code>[3], [1;5],<br/>'a', 'b', 'c']</code> |
| <code>[3] o (3::<br/>[])</code> | <code>[3]</code>                                           | <code>[], [1;5],<br/>'a', 'b', 'c']</code>  |
| <code>3::x</code>               | <code>[3], [3;4;5]</code>                                  | <code>[], [1;5],<br/>'a', 'b', 'c']</code>  |
| <code>x::y</code>               | <code>[3], [3;4;5], [1;5], ['a', 'b', 'c']</code>          | <code>[]</code>                             |
| <code>_</code>                  | <code>5, true, "abc", (3, "hello"),<br/>[3;2;4], []</code> |                                             |

Esempi con tipi di dato di base:

```
In [24]: let negazione b =  
         match b with  
         | true -> false  
         | false -> true ;;
```

```
Out[24]: val negazione : bool -> bool = <fun>
```

```
In [25]: let rec fibonacci n =  
         match n with  
         | 0 -> 0  
         | 1 -> 1  
         | _ -> fibonacci (n-1) + fibonacci (n-2);;
```

```
Out[25]: val fibonacci : int -> int = <fun>
```

La wildcard `_` messa in fondo cattura tutti gli altri casi

Qualche esempio con le tuple:

```
In [26]: let my_or x y =  
         match (x,y) with  
         | (false,false) -> false  
         | _ -> true;;
```

```
Out[26]: val my_or : bool -> bool -> bool = <fun>
```

```
In [27]: let first_true t =  
         match t with  
         | (true,_,_) -> 1  
         | (false,true,_) -> 2  
         | (false,false,true) -> 3  
         | (false,false,false) -> -1;;
```

```
Out[27]: val first_true : bool * bool * bool -> int = <fun>
```

Qualche esempio con le liste:

```
In [28]: let is_empty lis =  
         match lis with  
         | [] -> true  
         | _ -> false
```

```
Out[28]: val is_empty : 'a list -> bool = <fun>
```

```
In [29]: let inizia_con_zero lis =  
         match lis with  
         | [] -> false  
         | 0::lis' -> true  
         | x::lis' -> false
```

```
Out[29]: val inizia_con_zero : int list -> bool = <fun>
```

```
In [30]: let lunghezza_uno lis =  
         match lis with  
         | [] -> false
```

```
| x::[] -> true
| x::lis' -> false
```

Out[30]: val lunghezza\_uno : 'a list -> bool = <fun>

I casi `false` negli ultimi due esempi possono essere accorpati in `_`

È opportuno sottolineare che nel pattern `x::lis'` avremo sempre che `x` è un elemento (il primo) della lista che si sta facendo match, mentre `lis'` è la *lista* degli elementi che seguono. Quindi, nel caso di una lista di interi, `x` avrà tipo `int` mentre `lis'` avrà tipo `int list`.

## Pattern matching ed esaustività

È opportuno (ma non obbligatorio) che i pattern siano *esaustivi*:

- per qualunque valore ci deve essere (almeno) un pattern con cui fa match

Esempio non esaustivo (warning a tempo di compilazione):

```
In [31]: let giorno n =
  match n with
  | 1 -> "lunedì" | 2 -> "martedì" | 3 -> "mercoledì"
  | 4 -> "giovedì" | 5 -> "venerdì"
  | 6 -> "sabato" | 7 -> "domenica"
```

```
File "[31]", line 2, characters 4-156:
2 | ....match n with
3 |       | 1 -> "lunedì" | 2 -> "martedì" | 3 -> "mercoledì"
4 |       | 4 -> "giovedì" | 5 -> "venerdì"
5 |       | 6 -> "sabato" | 7 -> "domenica"
Warning 8: this pattern-matching is not exhaustive.
Here is an example of a case that is not matched:
0
```

Out[31]: val giorno : int -> string = <fun>

A tempo di esecuzione:

va tutto bene fintanto che i valori passati alla funzione sono tra quelli "matchabili"

```
In [32]: giorno 7;;
```

Out[32]: - : string = "domenica"

altrimenti viene sollevata un'eccezione

```
In [33]: giorno 8;;
```

```
Exception: Match_failure ("[31]", 2, 4).
Called from file "toplevel/toploop.ml", line 208, characters 17-27
```

## Pattern matching e costrutto `function`



Come già detto in precedenza, il costrutto `function` è alternativo a `fun` nelle definizioni di funzioni. Rispetto a `fun` `function` ha la limitazione di prevedere un unico parametro.

```
In [84]: function x -> x + 1 ;;
```

```
Out[84]: - : int -> int = <fun>
```

L'utilità di `function` sta nel fatto che include un meccanismo di pattern matching sul suo (unico) parametro. Anziché scrivere:

```
In [86]: let isEmpty lis =
  match lis with
  | [] -> true
  | x::lis' -> false ;;
```

```
Out[86]: val isEmpty : 'a list -> bool = <fun>
```

Si può invece scrivere:

```
In [87]: let isEmpty = function
  | [] -> true
  | x::lis' -> false ;;
```

```
Out[87]: val isEmpty : 'a list -> bool = <fun>
```

andando quindi a definire un elenco di pattern al posto del parametro della funzione. (Infatti `lis` non è più nominato... dopo `function` ci sono direttamente i pattern.)

Se la funzione deve prevedere più di un parametro, `function` lavora solo sull'ultimo. Ossia la funzione:

```
In [89]: let is_first n lis =
  match lis with
  | [] -> false;
  | x::lis' -> x=n ;;
```

```
Out[89]: val is_first : 'a -> 'a list -> bool = <fun>
```

Può essere riscritta (visto che il parametro su cui si fa pattern matching è l'ultimo) come segue:

```
In [90]: let is_first n = function
  | [] -> false
  | x::lis' -> x=n ;;
```

```
Out[90]: val is_first : 'a -> 'a list -> bool = <fun>
```

## La vera potenza del Pattern Matching

Abbiamo usato il pattern matching come strumento di *selezione condizionale*:

- l'abbiamo usato un po' come il costrutto `switch` presente in molti altri linguaggi (ad es. JavaScript)
- rispetto ad uno `switch` (o a un `if`) ci ha consentito di esprimere condizioni sulla struttura (ad es. lista non vuota, usando il pattern `x::lis`)

La vera potenza del pattern matching è che consente di "smontare" le strutture dati:

- elaborare i singoli elementi di una tupla
- elaborare i singoli elementi di una lista
- estrarre una sottolista

Questo grazie alle variabili presenti nei pattern!

Quando un valore fa match con un pattern `P_i` che contiene variabili, quelle variabili vengono istanziate e sono utilizzabili nell'espressione `EXP_i`

```
In [34]: let primo t =
  match t with
  | (x,_) -> x ;;
```

```
Out[34]: val primo : 'a * 'b -> 'a = <fun>
```

```
In [35]: let somma t =
  match t with
  | (x,y) -> x+y ;;
```

```
Out[35]: val somma : int * int -> int = <fun>
```

Abbiamo già visto che, sfruttando il pattern matching implicito in `let`, queste funzioni possono essere scritte più semplicemente così:

```
In [36]: let primo (x,y) = x ;;
let somma (x,y) = x+y ;;
```

```
Out[36]: val primo : 'a * 'b -> 'a = <fun>
```

```
Out[36]: val somma : int * int -> int = <fun>
```

Un primo esempio con le liste:

```
In [37]: let testa lis =
  match lis with
  | x::lis' -> x ;;
```

```
File "[37]", line 2, characters 4-37:
2 | ....match lis with
3 |     | x::lis' -> x...
Warning 8: this pattern-matching is not exhaustive.
Here is an example of a case that is not matched:
[]
```

```
Out[37]: val testa : 'a list -> 'a = <fun>
```

A causa del pattern matching non esaustivo, questa è una *funzione parziale*

- funziona solo su liste non vuote

## Accorpare casi del pattern matching

E' possibile accorpare casi del pattern matching per associarli alla stessa espressione

```
In [38]: let giorno_festivo n =
  match n with
  | 1 | 2 | 3 | 4 | 5 -> "feriale"
  | 6 | 7 -> "festivo"
  | _ -> "ERRORE" ;;
```

```
Out[38]: val giorno_festivo : int -> string = <fun>
```

Ma se si usano variabili, bisogna che siano presenti in tutti i pattern accorpati

```
In [39]: let rec somma_coppia c =
  match c with
  | (x,0) | (0,x) -> x
  | (x,y) -> x+y ;;
```

```
Out[39]: val somma_coppia : int * int -> int = <fun>
```

Altrimenti bisogna separare i casi, o sostituire le variabili (se non servono) con `_`

```
In [40]: let lunghezza_uno_due lis =
  match lis with
  | x::[] | x::y::[] -> true
  | _ -> false ;;
```

File "[40]", line 3, characters 6-22:

```
3 |   | x::[] | x::y::[] -> true
    ^^^^^^^^^^^^^^^^^^^
```

Error: Variable y must occur on both sides of this | pattern

In questo esempio sono stati accorpati i pattern `x::[]` e `x::y::[]`, e il problema è che la variabile `y` è usata solo in uno dei due casi pattern. Questo non piace al compilatore, in quanto `y` potrebbe essere usata nell'espressione associata a questi due pattern, e se così fosse quando la lista ha lunghezza uno l'espressione si troverebbe ad usare una variabile a cui non è associato alcun valore.

Ad esempio, se avessimo avuto:

```
| x::[] | x::y::[] -> x+y
```

nel caso in cui la lista avesse fatto match con `x::[]`, la variabile `y` nell'espressione `x+y` non sarebbe stata definita.

Nell'esempio della funzione `lunghezza_uno_due` in realtà questo problema non accade, in quanto l'espressione associata ai pattern è semplicemente `true`, e quindi non usa nessuna variabile. Il compilatore, però, effettua i controlli sulle variabili nei pattern senza tenere conto dell'espressione a destra di `->`, e per questo segnala l'errore.

Per risolvere la situazione, una prima soluzione è di separare i pattern in due casi distinti:

```
In [41]: let lunghezza_uno_due lis =
  match lis with
  | x::[] -> true
  | x::y::[] -> true
  | _ -> false ;;
```

```
Out[41]: val lunghezza_uno_due : 'a list -> bool = <fun>
```

Una seconda soluzione, visto che `x` e `y` in realtà non servono a nulla, è di utilizzare la wildcard `_` al posto delle variabili, come segue:

```
In [42]: let lunghezza_uno_due lis =
  match lis with
  | _::[] | _::_::[] -> true
  | _ -> false ;;
```

```
Out[42]: val lunghezza_uno_due : 'a list -> bool = <fun>
```

## Funzioni ricorsive su liste

Il pattern matching ci consente ora di scrivere funzioni (ricorsive) che processano liste

```
In [43]: let rec length lis =
  match lis with
  | [] -> 0
  | x::lis' -> 1 + length lis' ;;
```

```
Out[43]: val length : 'a list -> int = <fun>
```

```
In [44]: let rec somma lis =
  match lis with
  | [] -> 0
  | x::lis' -> x + somma lis' ;;
```

```
Out[44]: val somma : int list -> int = <fun>
```

```
In [45]: let rec contains x lis =
  match lis with
  | [] -> false
  | y::lis' -> if y=x then true
               else contains x lis'
```

```
Out[45]: val contains : 'a -> 'a list -> bool = <fun>
```

Implementazione di `@`:

```
In [46]: let rec append l1 l2 =
  match l1 with
  | [] -> l2
  | x :: l1' -> x :: (append l1' l2)
```

```
Out[46]: val append : 'a list -> 'a list -> 'a list = <fun>
```

Richiede tempo lineare nella lunghezza di `lis`

Rovesciare una lista

Soluzione poco efficiente (perché usa `@`):

```
In [47]: let rec rev lis =
  match lis with
  | [] -> []
  | x::lis' -> (rev lis') @ [x] ;;
```

```
Out[47]: val rev : 'a list -> 'a list = <fun>
```

Soluzione più efficiente che usa un parametro `lis2` di accumulazione del risultato (*accumulatore*):

```
In [48]: let rev lis =
  let rec rev_accum lis1 lis2 =
    match lis1 with
    | [] -> lis2
    | x::lis1' -> rev_accum lis1' (x::lis2)
  in
  rev_accum lis [] ;;
```

```
Out[48]: val rev : 'a list -> 'a list = <fun>
```

La funzione `rev_accum` ad ogni chiamata ricorsiva prende l'elemento in testa alla lista `lis1` e lo "sposta" in testa alla lista `lis2`. Quindi gli elementi vengono presi uno dopo l'altro da `lis1` e aggiunti uno prima dell'altro in `lis2`, con il risultato di rovesciare la lista.

In questo esempio il parametro `lis2` è detto *accumulatore* in quanto è un parametro che viene usato per costruire passo passo il risultato. Nell'ambito della programmazione funzionale, in cui la ricorsione è fondamentale e largamente usata, i parametri di accumulazione vengono spesso utilizzati in quanto consentono di definire funzioni con la ricorsione in coda (*tail-recursive*), ossia in cui la chiamata ricorsiva è l'ultima operazione svolta.

Questo è il caso anche della funzione `rev_accum`, in cui, nel caso ricorsivo, la cui chiamata ricorsiva è l'ultima operazione effettivamente svolta, dopo aver creato il risultato parziale `x::lis2` nel parametro di accumulazione.

L'utilizzo della tail-recursion è molto importante nelle funzioni che prevedono una lunga sequenza di chiamate ricorsive. Essa infatti, grazie ad ottimizzazioni che vengono svolte a tempo di esecuzione, consente di allocare nello stack un unico record di attivazione "riciclandolo" ad ogni chiamata ricorsiva. Come conseguenza, il programma viene eseguito più velocemente (si risparmia il tempo di allocare un nuovo record di attivazione), consumando meno memoria e, soprattutto, senza correre il rischio di riempire tutta la memoria a disposizione dello stack (*stack overflow*) con un conseguente arresto del programma.

## Un esempio non banale: area di un poligono irregolare

Scriviamo un programma per calcolare l'area di un qualunque poligono usando il metodo descritto qui:

- <https://www.wikihow.it/Calcolare-l'Area-di-un-Poligono>



In sostanza, il metodo prevede di:

- prendere i vertici *in senso antiorario* come lista di coordinate cartesiane  $(x_1, y_1), \dots, (x_n, y_n)$ , copiando il primo vertice in fondo alla lista:

```
[ (-3,-2); (-1,4); (6,1); (3,10); (-4,9); (-3,-2) ]
```

- calcolare la somma dei prodotti di ogni  $x_i$  con  $y_{i+1}$

```
(-3*4) + (-1*1) + (6*10) + (3*9) + (-4*-2) = 82
```

- calcolare la somma dei prodotti di ogni  $y_i$  con  $x_{i+1}$

```
(-2*-1) + (4*6) + (1*3) + (10*-4) + (9*-3) = -38
```

- sottrarre il secondo dal primo e dividere per due

```
(82-(-38))/2 = (82+38)/2 = 120/2 = 60
```

Soluzione completa:

```
In [49]: let poligono = [ (-3,-2); (-1,4); (6,1); (3,10); (-4,9) ] ;;

let area pol =
  let rec passo1 (x0,y0) lis =
    match lis with
    | [] -> 1 (* non viene mai eseguito *)
    | (x,y)::[] -> x*y0
    | (x1,y1)::(x2,y2)::lis' -> x1*y2 + passo1 (x0,y0) ((x2,y2)::lis')
  in
  let rec passo2 (x0,y0) lis =
    match lis with
    | [] -> 1 (* non viene mai eseguito *)
    | (x,y)::[] -> y*x0
    | (x1,y1)::(x2,y2)::lis' -> y1*x2 + passo2 (x0,y0) ((x2,y2)::lis')
  in
  match pol with
  | [] -> 0.
  | p0::lis -> float_of_int (passo1 p0 pol - passo2 p0 pol) /. 2. ;;
  (* invece che copiare il primo elemento in fondo, lo passiamo come parametro *)
area poligono;;
```

```

Out[49]: val poligono : (int * int) list =
          [(-3, -2); (-1, 4); (6, 1); (3, 10); (-4, 9)]
Out[49]: val area : (int * int) list -> float = <fun>
Out[49]: - : float = 60.

```

Questa funzione esegue le due sommatorie previste dal metodo tramite due funzioni ricorsive `passo1` e `passo2`. Queste due funzioni dovrebbero lavorare sulla lista in cui il primo elemento è stato copiato in fondo. Questo richiederebbe innanzitutto di eseguire `pol @ [p0]`, dove `p0=(x0,y0)` è il primo elemento di `pol`. Questo però sarebbe inefficiente e consumerebbe memoria (per copiare tutto `pol`). La soluzione adottata, invece, passa `p0` come parametro a `passo1` e `passo2`. Queste due funzioni continueranno a passarsi `p0` da una chiamata all'altra mentre scorrono la lista. Arrivate in fondo alla lista utilizzeranno il `p0` ricevuto come parametro come se fosse l'ultimo elemento della lista.

Le funzioni `passo1` e `passo2` sono sostanzialmente identiche. L'unica cosa che cambia è che la prima ad ogni passo calcola `x1*y2`, mentre la seconda calcola `x2*y1`.

L'espressione che avvia il calcolo è quella nell'ultima riga del programma. Se `pol` non è vuota, tramite il pattern `p0::lis` viene identificato il primo elemento di tale lista e viene utilizzato per chiamare `passo1` e `passo2`. Viene eseguita la sottrazione tra i risultati ottenuti dalle due funzioni e poi viene eseguita la divisione per due.

Mentre `passo1`, `passo2` e la sottrazione vengono eseguite lavorando su dati di tipo `int` (nota: le coordinate dei punti nella lista sono dati come valori interi) la divisione per due viene eseguita ricorrendo all'operazione su `float`. Questo perché a differenza delle somme, moltiplicazioni e sottrazioni usate in `passo1` e `passo2`, l'operazione di divisione su interi `/` potrebbe introdurre un'approssimazione (l'area potrebbe non essere intera). Per questo motivo è più corretto usare `/.` dopo avere eseguito l'opportuna conversione da `int` a `float`.

Vediamo ora però come migliorare questa soluzione evitando di scandire due volte la lista (una volta per `passo1` e una volta per `passo2`).

Soluzione migliorata (accorpa `passo1` e `passo2` in un'unica funzione ricorsiva):

```

In [50]: let poligono = [ (-3,-2); (-1,4); (6,1); (3,10); (-4,9) ] ;;

let area pol =
  let rec calcolo (x0,y0) lis =
    match lis with
    | [] -> (1,1) (* non viene mai eseguito *)
    | (x,y)::[] -> (x*y0 , y*x0)
    | (x1,y1)::(x2,y2)::lis' ->
      let (ris1,ris2) = calcolo (x0,y0) ((x2,y2)::lis')
      in (ris1 + x1*y2 , ris2 + y1*x2)
  in
  match pol with
  | [] -> 0.
  | p0::lis -> let (ris1,ris2) = calcolo p0 pol
               in float_of_int (ris1 - ris2) /. 2. ;;

```

```
area poligono;;
```

```
Out[50]: val poligono : (int * int) list =
          [(-3, -2); (-1, 4); (6, 1); (3, 10); (-4, 9)]
Out[50]: val area : (int * int) list -> float = <fun>
Out[50]: - : float = 60.
```

In questa soluzione le funzioni `passo1` e `passo2` sono state sostituite da un'unica funzione ricorsiva `calcolo` che esegue i due conteggi scandendo un'unica volta la lista e portandosi dietro una coppia di valori, anziché un solo valore. Nel primo elemento della coppia verrà calcolato il risultato che corrispondeva a `passo1`, e nel secondo quello che corrispondeva a `passo2`.

## Funzioni higher-order su liste

Abbiamo visto che nella programmazione funzionale la ricorsione è l'unico modo con cui possiamo ripetere un calcolo più volte

- Per scandire o elaborare una lista è inevitabile ricorrere alla ricorsione
- Esistono però degli schemi ricorrenti nelle funzioni che processano liste...

Questi schemi possono essere modellati come utili funzioni higher-order

Vediamo qualche esempio:

```
In [51]: let rec contiene_zero lis =
          match lis with
          | [] -> false
          | x::lis' -> if x=0 then true
                      else contiene_zero lis' ;;
```

```
Out[51]: val contiene_zero : int list -> bool = <fun>
```

```
In [52]: let rec contiene_positivo lis =
          match lis with
          | [] -> false
          | x::lis' -> if x>0 then true
                      else contiene_positivo lis' ;;
```

```
Out[52]: val contiene_positivo : int list -> bool = <fun>
```

```
In [53]: let rec contiene_pari lis =
          match lis with
          | [] -> false
          | x::lis' -> if x mod 2 = 0 then true
                      else contiene_pari lis' ;;
```

```
Out[53]: val contiene_pari : int list -> bool = <fun>
```

Tutte queste funzioni:

- scandiscono la lista (usando la ricorsione)



- valutano una *condizione* su ogni elemento (essere uguale a `x`, essere positivo, essere pari, ...)
- restituiscono `true` se incontrano un elemento in cui la condizione è vera
- restituiscono `false` se arrivano in fondo alla lista

L'unica differenza tra le tre funzioni `contiene_zero`, `contiene_positivo` e `contiene_pari` è nella condizione testata su ogni elemento

### IDEA:

- astraiamo il test della condizione come un *predicato* (funzione `XXX -> bool`)
- scriviamo un'unica funzione che prende tale predicato come parametro e verifica se ESISTA un elemento che lo soddisfi

## Exists

Ecco la funzione (higher-order) che rappresenta questo schema

```
In [54]: let rec exists p lis =
  match lis with
  | [] -> false
  | x::lis' -> if p x then true
               else exists p lis' ;;
```

```
Out[54]: val exists : ('a -> bool) -> 'a list -> bool = <fun>
```

ora possiamo ridefinire le funzioni `contiene_zero`, `contiene_positivo` e `contiene_pari` usando la ricorsione in modo implicito (nascosta dentro a `exists`)

```
In [55]: let contiene_zero lis = exists (fun x -> x=0) lis ;;
let contiene_positivo lis = exists (fun x -> x>0) lis ;;
let contiene_pari lis = exists (fun x -> x mod 2 = 0) lis ;;
```

```
Out[55]: val contiene_zero : int list -> bool = <fun>
```

```
Out[55]: val contiene_positivo : int list -> bool = <fun>
```

```
Out[55]: val contiene_pari : int list -> bool = <fun>
```

Anche la funzione `contains`, che verifica se un certo elemento è presente, può essere espressa in termini di `exists`:

```
In [56]: let contains x lis = exists (fun y -> x=y) lis ;;
```

```
Out[56]: val contains : 'a -> 'a list -> bool = <fun>
```

In questo caso la funzione `exists` riceve la *chiusura* del predicato

- include il valore corrente di `x`

Vediamo un altro modo "creativo" di definire `contains` con il paradigma funzionale

- usando un'applicazione parziale di funzione

```
In [57]: let uguale x y = x=y;;
let contains x lis = exists (uguale x) lis;;
```

```
Out[57]: val uguale : 'a -> 'a -> bool = <fun>
```

```
Out[57]: val contains : 'a -> 'a list -> bool = <fun>
```

## Forall

In modo simile possiamo definire una funzione higher-order che testa un predicato su *tutti* gli elementi della lista

```
In [58]: let rec forall p lis =
  match lis with
  | [] -> true
  | x::lis' -> if p x then forall p lis'
               else false ;;
```

```
Out[58]: val forall : ('a -> bool) -> 'a list -> bool = <fun>
```

e possiamo definire le funzioni `tutti_zeri`, `tutti_positivi` e `tutti_pari`

```
In [59]: let tutti_zeri lis = forall (fun x -> x=0) lis ;;
let tutti_positivi lis = forall (fun x -> x>0) lis ;;
let tutti_pari lis = forall (fun x -> x mod 2 = 0) lis ;;
```

```
Out[59]: val tutti_zeri : int list -> bool = <fun>
```

```
Out[59]: val tutti_positivi : int list -> bool = <fun>
```

```
Out[59]: val tutti_pari : int list -> bool = <fun>
```

## Filter

Un'altra operazione frequente sulle liste è quella di selezionare (filtrare) gli elementi secondo una condizione

- restituendo la lista degli elementi che la soddisfano

Anche in questo caso, possiamo definire una funzione higher-order che astrae la condizione con un predicato

```
In [60]: let rec filter p lis =
  match lis with
  | [] -> []
  | x::lis' -> if p x then x::filter p lis'
               else filter p lis' ;;
```

```
Out[60]: val filter : ('a -> bool) -> 'a list -> 'a list = <fun>
```

Questa volta la funzione restituisce una lista

Possiamo usare `filter` per definire le funzioni `estrai_zeri`, `estrai_positivi` e `estrai_pari`, sempre usando la ricorsione in modo implicito.

```
In [61]: let estrai_zeri lis = filter (fun x -> x=0) lis ;;
let estrai_positivi lis = filter (fun x -> x>0) lis ;;
let estrai_pari lis = filter (fun x -> x mod 2 = 0) lis ;;
```

```
Out[61]: val estrai_zeri : int list -> int list = <fun>
```

```
Out[61]: val estrai_positivi : int list -> int list = <fun>
```

```
Out[61]: val estrai_pari : int list -> int list = <fun>
```

## Map

Altra elaborazione frequente: applicare la stessa operazione a tutti gli elementi

- produce una nuova lista con tanti elementi quanto quella processata
- il tipo degli elementi può essere diverso
- per astrarre sull'operazione abbiamo bisogno di una funzione, non di un predicato

```
In [62]: let rec map f lis =
  match lis with
  | [] -> []
  | x::lis' -> f x::map f lis' ;;
```

```
Out[62]: val map : ('a -> 'b) -> 'a list -> 'b list = <fun>
```

Qualche esempio d'uso:

```
In [63]: map (fun x -> x+1) [1;2;3] ;;
```

```
Out[63]: - : int list = [2; 3; 4]
```

```
In [64]: let primo lis = map (fun (x,y) -> x) lis ;;
primo [ (3,2); (4,7); (9,2) ] ;;
```

```
Out[64]: val primo : ('a * 'b) list -> 'a list = <fun>
```

```
Out[64]: - : int list = [3; 4; 9]
```

## Fold-right

Le funzioni higher-order viste fino ad ora lavoravano sui singoli elementi della lista in modo indipendente:

- `filter` valuta ogni elemento e sceglie se inserirlo nel risultato (indipendentemente dagli altri)
- `map` applica una funzione ad ogni elemento (indipendentemente dagli altri)
- ...

Spesso si vuole elaborare tutti gli elementi della lista per calcolare un unico risultato

Ad esempio:

- Calcolare la *somma* degli elementi di una lista
- Calcolare *minimo e massimo* di una lista

- *Concatenare tutti gli elementi* di una lista di stringhe

Intuitivamente, queste elaborazioni richiedono di scandire la lista portandosi dietro una o più variabili che memorizzano il *risultato parziale* via via calcolato

Ad esempio, nell'approccio imperativo (in JavaScript e con array...) useremmo un `for` :

```
function somma(a) {
  var s = 0
  for (var i in a)
    s += a[i]
  return s
}
```

Lo stesso per calcolare minimo e massimo, concatenare tutti gli elementi, ecc...

L'importante è capire:

- che variabili "portarsi dietro" nel ciclo
- come aggiornarle ad ogni iterazione

Anche seguendo un approccio *ricorsivo*, nella scansione di una lista è fondamentale capire come portarsi dietro il risultato parziale e come aggiornarlo ad ogni passo

Il risultato parziale non sarà memorizzato in una variabile, ma passato da una chiamata all'altra come parametro o valore di ritorno

```
In [65]: let rec somma lis =
  match lis with
  | [] -> 0
  | x::lis' -> x + somma lis' ;;
somma [3;2;4] ;;
```

```
Out[65]: val somma : int list -> int = <fun>
```

```
Out[65]: - : int = 9
```

In questo esempio, ogni chiamata a `somma` restituisce la somma della porzione di lista che va dall'elemento corrente fino in fondo

- il risultato parziale è restituito dalla funzione

Vediamo che cosa succede nello *stack*:

Somma\_ric\_1

A cui segue:

Somma\_ric\_2

Quindi:

- gli elementi della lista vengono in realtà sommati dall'ultimo al primo (da *destra*)

- ad ogni passo si applica la funzione/operatore `(+)` che somma due numeri

Cerchiamo di estrarre lo schema ricorsivo su cui si basa la funzione `somma`

```
somma [3;2;4]
3 + somma [2;4]
3 + (2 + somma [4])
3 + (2 + (4 + somma []))
3 + (2 + (4 + 0))
```

Generalizziamo `(+) = f`

```
f 3 (f 2 (f 4 0))
```

Generalizziamo `lis = [x1;x2;...;xN]` e caso base restituisce `a` invece che `0`

```
f x1 (f x2 (... (f xN a)...))
```

La funzione `fold_right` realizza lo schema ricorsivo che abbiamo identificato

```
In [66]: let rec fold_right f lis a =
          match lis with
          | [] -> a
          | x::lis' -> f x (fold_right f lis' a) ;;
```

```
Out[66]: val fold_right : ('a -> 'b -> 'b) -> 'a list -> 'b -> 'b = <fun>
```

Prevede i parametri (oltre alla lista `lis`):

- `f`: la funzione da applicare ad ogni passo
- `a`: il risultato corrispondente al caso base (lista vuota `[]`)

Ad ogni passo, applica `f` all'elemento e al risultato ottenuto sul resto della lista

Usiamo `fold_right` per definire `somma` usando la ricorsione in modo implicito:

```
In [67]: let somma lis = fold_right (+) lis 0 ;;
          somma [3;2;4] ;;
```

```
Out[67]: val somma : int list -> int = <fun>
```

```
Out[67]: - : int = 9
```

Altri esempi d'uso di `fold_right`:

```
In [68]: let concat lis = fold_right (^) lis "" ;;
          concat ["abc"; "def"; "gh"] ;;
```

```
Out[68]: val concat : string list -> string = <fun>
```

```
Out[68]: - : string = "abcdefgh"
```

```
In [69]: let stringa_piu_lunga lis =
  let f x s =
    if String.length x > String.length s then x else s
  in
  fold_right f lis "" ;;

stringa_piu_lunga ["ciao";"hello";"hi"] ;;
```

```
Out[69]: val stringa_piu_lunga : string list -> string = <fun>
```

```
Out[69]: - : string = "hello"
```

Calcolare minimo e massimo di una lista

- che cosa è utile portarsi dietro durante la scansione della lista?
- che funzione si deve applicare ad ogni passo?
- inoltre: qual è il valore del caso base `[]` ??

Idea: si estrae il primo elemento e lo si usa come caso base

```
In [70]: let minimo_massimo lis =
  let f x (min,max) =
    if x < min then (x,max)
    else if x > max then (min,x)
    else (min,max)
  in
  match lis with
  | [] -> (0,0)
  | x::lis' -> fold_right f lis' (x,x) ;;

minimo_massimo [3;2;4] ;;
```

```
Out[70]: val minimo_massimo : int list -> int * int = <fun>
```

```
Out[70]: - : int * int = (2, 4)
```

In questa implementazione della funzione `minimo_massimo` viene restituita la coppia `(0,0)` nel caso in cui la lista sia vuota. Dal momento che il primo elemento viene estratto ed usato come primo candidato minimo e massimo, la funzione ricorsiva viene utilizzata su `lis'`, quindi la ricorsione parte a tutti gli effetti dal secondo elemento della lista.

## Fold-left

Tornando all'esempio della somma, si può definire una funzione ricorsiva anche così:

```
In [71]: let rec somma a lis =
  match lis with
  | [] -> a
  | x::lis' -> somma (a+x) lis' ;;

somma 0 [3;2;4] ;;
```

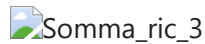
```
Out[71]: val somma : int -> int list -> int = <fun>
```

Out[71]: - : int = 9

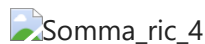
Il risultato parziale viene:

- *passato come parametro* (inizialmente zero)
- aggiornato man mano che si incontrano nuovi elementi scandendo la lista dall'inizio alla fine (da *sinistra*)

Vediamo che cosa succede nello *stack*:

Somma\_ric\_3

A cui segue:

Somma\_ric\_4

Quindi:

- gli elementi della lista vengono sommati dal primo all'ultimo (da *sinistra*)
- ad ogni passo si applica la funzione/operatore `(+)` che somma due numeri
- il risultato "scende" immutato lungo lo stack (la funzione è *tail recursive*, quindi l'uso dello stack si può ottimizzare...)

Come già detto nel capitolo precedente, il fatto che la funzione sia abbia la chiamata ricorsiva in coda (tail-recursive) fa sì che in realtà venga utilizzato un unico record di attivazione nello stack. Questo minimizza l'uso della memoria e previene il problema dello *stack overflow*.

Estraiamo lo schema ricorsivo su cui si basa la nuova versione della funzione `somma`

```
somma 0 [3;2;4]
somma (0 + 3) [2;4]
somma ((0 + 3) + 2) [4]
somma (((0 + 3) + 2) + 4) []
(((0 + 3) + 2) + 4)
```

Generalizziamo `(+) = f`

```
f (f (f 0 3) 2) 4
```

Generalizziamo ancora `lis = [x1;x2;...;xN]` e valore iniziale `a` invece che `0`

```
f (f (... (f (f a x1) x2) ...) xN-1) xN
```

La funzione `fold_left` realizza lo schema ricorsivo che abbiamo identificato

```
In [72]: let rec fold_left f a lis =
  match lis with
  | [] -> a
  | x::lis' -> fold_left f (f a x) lis' ;;
```

```
Out[72]: val fold_left : ('a -> 'b -> 'a) -> 'a -> 'b list -> 'a = <fun>
```

Prevede i parametri (oltre alla lista `lis`):

- `f` : la funzione da applicare ad ogni passo
- `a` : il valore iniziale da cui partire con il calcolo

Ad ogni passo, richiama ricorsivamente `f` passandogli un valore via via aggiornato

Ora possiamo usare `fold_left` per definire `somma` usando la ricorsione in modo implicito

```
In [73]: let somma lis = fold_left (+) 0 lis ;;
        somma [3;2;4] ;;
```

```
Out[73]: val somma : int list -> int = <fun>
```

```
Out[73]: - : int = 9
```

Altri esempi d'uso di `fold_left` (gli stessi già visti con `fold_right`):

```
In [74]: let concat lis = fold_left (^) "" lis ;;
        concat ["abc"; "def"; "gh"] ;;
```

```
Out[74]: val concat : string list -> string = <fun>
```

```
Out[74]: - : string = "abcdefgh"
```

```
In [75]: let stringa_piu_lunga lis =
        let f s x =
            if String.length x > String.length s then x else s
        in
        fold_left f "" lis ;;

        stringa_piu_lunga ["ciao";"hello";"hi"] ;;
```

```
Out[75]: val stringa_piu_lunga : string list -> string = <fun>
```

```
Out[75]: - : string = "hello"
```

Calcolare minimo e massimo di una lista:

```
In [76]: let minimo_massimo lis =
        let f (min,max) x =
            if x < min then (x,max)
            else if x > max then (min,x)
            else (min,max)
        in
        match lis with
        | [] -> (0,0)
        | x::lis' -> fold_left f (x,x) lis' ;;

        minimo_massimo [3;2;4] ;;
```

```
Out[76]: val minimo_massimo : int list -> int * int = <fun>
```



Out[76]: - : int \* int = (2, 4)

**ATTENZIONE:** rispetto alla versione con `fold_right`, i parametri di `f` sono invertiti e anche quelli di `fold_left` hanno un ordine diverso

## Fold-right VS Fold-left

Ma quindi `fold_right` e `fold_left` sono intercambiabili?

- No!

Innanzitutto:

- `fold_left` è tail recursive (quindi si può ottimizzare l'uso dello stack)
- `fold_right` non lo è (anche se esistono implementazioni tail recursive)

Ma soprattutto, in alcuni casi possono portare a risultati diversi!

Esempio: funzione identità su liste (fa una "copia" della lista)

```
In [77]: let id_lista1 lis =
           let f x l = x::l
           in fold_right f lis [];;

let id_lista2 lis =
  let f l x = x::l
  in fold_left f [] lis;;

id_lista1 [1;2;3] ;;
id_lista2 [1;2;3] ;;
```

Out[77]: val id\_lista1 : 'a list -> 'a list = <fun>

Out[77]: val id\_lista2 : 'a list -> 'a list = <fun>

Out[77]: - : int list = [1; 2; 3]

Out[77]: - : int list = [3; 2; 1]

La versione con `fold_left` ha **ROVESCIATO** la lista

- `::` aggiunge in testa! Per copiare la lista gli elementi gli vanno dati dall'ultimo al primo

## Un paio di esempi con `fold_right` e `fold_left`

- Crea una nuova lista sommando gli elementi a partire dal fondo

```
In [78]: let somma_dal_fondo lis =
           let f x l =
             match l with
             | [] -> [x]
             | x'::l' -> x+x'::l
```

```
in
  fold_right f lis [] ;;
```

```
Out[78]: val somma_dal_fondo : int list -> int list = <fun>
```

```
In [79]: somma_dal_fondo [2;2;2;2;2] ;;
```

```
Out[79]: - : int list = [10; 8; 6; 4; 2]
```

- verifica se una lista è ordinata in modo crescente

```
In [80]: let crescente lis =
  (* f si "porta dietro" il precedente *)
  let f (prec,b) x = (x,b && prec<=x)
  in match lis with
  | [] -> true
  | x::lis' -> let (_,b) =
                fold_left f (x,true) lis'
              in b;;
```

```
Out[80]: val crescente : 'a list -> bool = <fun>
```

```
In [81]: crescente [1;4;4;6;9] ;;
crescente [3;5;4;7;9] ;;
```

```
Out[81]: - : bool = true
```

```
Out[81]: - : bool = false
```

Quest'ultimo esempio mostra come sia possibile elaborare ad ogni passo due elementi consecutivi della lista. Questo di base non è previsto da `fold_right` e `fold_left`, in quanto la funzione ausiliaria `f` da loro utilizzata legge un solo elemento della lista per volta. E' però possibile aggiungere tra i dati da portarsi dietro per il calcolo un'informazione sul precedente valore della lista incontrato ( `prec`, nell'esempio) facendolo aggiornare di volta in volta alla `f`.

Nel caso di `fold_left`, che scandisce gli elementi dal primo all'ultimo, questo `prec` sarà effettivamente l'elemento che nella lista precede l'elemento corrente. Nel caso di `fold_right`, questo `prec` sarà invece l'elemento successivo a quello corrente, dal momento `fold_right` scandisce la lista in direzione inversa.

## Le funzioni su liste nel modulo `List`

Le funzioni higher-order su liste che abbiamo visto sono disponibili nel modulo `List` della libreria standard di OCaml (<https://ocaml.org/api/List.html>)

- `exists`  $\Rightarrow$  `List.exists`
- `forall`  $\Rightarrow$  `List.for_all`
- `filter`  $\Rightarrow$  `List.filter`
- `map`  $\Rightarrow$  `List.map`
- `fold_right`  $\Rightarrow$  `List.fold_right`

- `fold_left`  $\Rightarrow$  `List.fold_left`

Oltre a molte altre funzioni utili:

- `List.find` , `List.sort` , `List.partition` , `List.merge` ,...

## Esempio: area di un poligono irregolare con `fold_right` e `List`

Riprendendo l'esempio del poligono irregolare:

```
In [82]: let poligono = [ (-3,-2); (-1,4); (6,1); (3,10); (-4,9) ] ;;

let area pol =
  let f (x1,y1) ((x2,y2),(ris1,ris2)) =
    ((x1,y1),(ris1+x1*y2,ris2+x2*y1))
  in
  let (p,(ris1,ris2)) = List.fold_right f pol (List.hd pol,(0,0))
  in
  (float_of_int (ris1 - ris2)) /. 2.;;

area poligono;;
```

```
Out[82]: val poligono : (int * int) list =
  [(-3, -2); (-1, 4); (6, 1); (3, 10); (-4, 9)]
```

```
Out[82]: val area : (int * int) list -> float = <fun>
```

```
Out[82]: - : float = 60.
```

Questa implementazione della funzione `area` è molto più compatta di quelle date in precedenza, ma anche più difficile da comprendere a primo sguardo.

Si usa `fold_right` per scandire la lista dalla coda alla testa. La funzione `f` che viene richiamata ad ogni passo prende come primo parametro l'elemento corrente (la coppia `(x1,y1)`) e i dati da "portarsi dietro" che includono l'elemento successivo a quello corrente `(x2,y2)` e i risultati parziali calcolati fino a quel momento `(ris1,ris2)`. Dal momento che i dati da portarsi dietro devono essere passati come unico parametro, queste due coppie diventano una coppia di coppie `((x2,y2),(ris1,ris2))`.

La chiamata a `fold_right` inizializza queste informazioni a `List.hd` (il primo elemento della lista che, concettualmente, dovrebbe essere copiato in fondo, ma che invece sarà elaborato senza effettivamente aggiungerlo alla lista) e `(0,0)` in quanto il calcolo deve ancora iniziare.

Ad ogni passo, `f` prende l'elemento corrente (dall'ultimo al primo) fa le moltiplicazioni con il successivo (`x1*y2` e `x2*y1`) e somma quanto ottenuto ai risultati parziali `ris1` e `ris2`. Il risultato di `f`, oltre a contenere i valori aggiornati di `ris1` e `ris2`, conterrà come primo elemento della coppia restituita, l'elemento corrente `(x1,y1)`, in modo che diventi, nella successiva chiamata di `f` operata dalla `fold_right`, l'elemento considerato come successivo.

Il risultato finale che sarà restituito da `fold_right` è una coppia `(p, (ris1, ris2))` in cui `p` corrisponderà al primo elemento della lista (che ci stiamo portando dietro ricorsivamente), mentre `ris1` e `ris2` saranno gli effettivi risultati corrispondenti alle due fasi di elaborazione viste all'inizio.

Il risultato della funzione `pol` sarà quindi `(float_of_int (ris1 - ris2)) /. 2.`, come nelle versioni della funzione `area` già illustrate in precedenza.

Questa funzione è stata implementata usando `fold_right` in modo da rispecchiare l'ordine di esecuzione delle operazioni che si otteneva eseguendo le precedenti versioni mostrate. È possibile definirne una versione usando invece `fold_left`.

## Funzioni higher-order su liste in altri linguaggi

Le funzioni higher-order su liste che abbiamo visto sono presenti (a volte parzialmente) anche in altri linguaggi.

- In JavaScript, ad esempio, funzioni quali `map`, `filter`, `reduce` (equivale a `fold_left`) possono essere usate su array

JAVASCRIPT:

```
const array1 = [1, 2, 3, 4];  
const reducer = (accum, current) => accum + current;  
console.log(array1.reduce(reducer)); // 1+2+3+4=10
```

- In Java, dalla versione 8 (del 2014), sono stati introdotti elementi di programmazione funzionale (lambda expressions) e metodi higher-order per alcune strutture dati
- ...